

Sección: PHP

1. Fundamentos

- **Pregunta:** ¿Cómo se puede averiguar el contenido y el tipo de una variable?
- **Respuesta:**

Creo que el 90% durante la fase de desarrollo si tengo que comprobar el contenido, por ejemplo, usando **CLI** siempre he utilizado **var_dump**, pero si agregamos más complejidad, por ejemplo, **print_r** es de lo mejor que hay, a menos que desarrollemos una función propia por supuesto, pero toma algo más de tiempo.

En las prácticas la documentación sistemática de funciones indicando siempre el tipo de **return** evitaba muchos errores siquiera antes de que fuesen posibles al detectar el paso equivocado de variables y sus valores en llamadas, argumentos, etc, sin embargo, como medida preventiva siempre podemos usar métodos nativos de **PHP** como **"is_string(\$variable)"**, **"is_int(\$variable)"**, etc.

2. Fundamentos avanzados

- **Pregunta:** Explica la diferencia entre *include*, *require*, *include_once* y *require_once* en PHP. ¿Cuándo usarías cada uno?
- **Respuesta:**

Antes de la respuesta me gustaría explicar que una de las diferencias principales entre ellos es el tipo de error nativo de **PHP** que ocasionan (no son excepciones, son errores), siendo estos los siguientes en la tabla:

MÉTODO	ERRORES DE PHP	COMPORTAMIENTO
include / include_once	E_WARNING	Detiene el script en seco.
require / require_once	E_ERROR	Advertencia de "error", pero continua

Sabiendo esto enlisto la respuesta de las diferencias entre cada uno:

- **include:** Permite incluir una o varias veces un archivo en el código, ¿En que escenarios es deseable poder incluir algo más de una vez? Por ejemplo, cuando el fichero sufre cambios o modificaciones durante el uso de la aplicación y se debe poder recargar/cargar otra vez con estos cambios, **EJEMPLO DE USO REAL:** Con mis alumnos diseño normalmente aplicaciones pequeñas con **MVC** como parte de sus asignaturas y es muy común usar **include** cuando por ejemplo mostramos una información inicial de un pedido, la aplicación

modifica el estado del pedido y el **include** incluye otra vez el mismo archivo reflejando los cambios, sin embargo, hay otras formas de hacer esto y normalmente no lo suelo utilizar, para **logs** lo he utilizado alguna vez, pero de nuevo, hay otras formas.

- **require:** Aquí la diferencia es el comportamiento cuando aquel archivo que se intenta cargar/renderizar no exista, no nos lanza una advertencia como **include** con **E_WARNING** y sigue ejecutando los scripts de todas formas si no que “mata el script” acabando con el flujo de la aplicación en seco, crítico para la carga de archivos esenciales, como conexiones a base de datos, archivos de configuración críticos, más allá de eso, tiene una serie de usos algo limitados porque son pocos los escenarios donde requieres más de una vez un archivo.
- **include_once:** El comportamiento es idéntico al del **include** normal previamente explicado pero no permite incluir/cargar/renderizar varias veces el mismo archivo/script, como se utiliza para la carga de componentes/script no esenciales es común verlo sobretodo en la carga de temas personalizados, header, footer, **EJEMPLO REAL DE USO:** En aquellos proyectos que he realizado estudiando o con mis alumnos cuando hacemos aplicaciones **MVC** suelo explicar su funcionamiento al intentar cargar un **banner publicitario** que si existe, se carga y si no, aunque nos arroja un error del tipo **E_WARNING** el script no se tiene y renderiza todo lo demás.
- **require_once:** El contexto de las explicaciones anteriores sirve para entender que hace lo mismo que el **require** pero solo incluye/renderiza una vez, este se usa en aquellos escenarios donde nos planteamos la pregunta de “¿Esto existe y ha sido cargado? Si la respuesta es no, detén todo, es crítico que esté disponible para el funcionamiento de la aplicación”, **EJEMPLO REAL DE USO:** Para aquellos scripts donde es obligatorio, crítico y fundamental importar **DTO** o conexiones a bases de datos.

Como valor añadido dejo una pequeña tabla que explica el tipo de retorno (si es que tiene) de los 4 casos de uso anteriormente explicados:

MÉTODO	ESCENARIO	RETORNO EN FALLO	RETORNO EN ÉXITO
include	Función con return	FALSE	El valor del return
	Función sin return	FALSE	Código de exito (int) “1”.
include_once	Función con return	FALSE	El valor del return
	Función sin return	FALSE	Código de exito (int) “1”.
require	Función con return	NADA (detiene el script)	El valor del return
	Función sin return	NADA (detiene el script)	Código de exito (int) “1”.

require once	Función con return	NADA (detiene el script)	El valor del return
	Función sin return	NADA (detiene el script)	Código de éxito (int) "1".

3. Estructuras de control

- **Pregunta:** Escribe un programa que simule un menú de opciones utilizando un switch o un while. El menú debe permitir al usuario elegir entre diferentes opciones (por ejemplo, calcular un área, convertir una temperatura, etc.). Implementa al menos tres opciones diferentes.
- **Respuesta:** Se que en algunas empresas está mal visto documentar o llenar de comentarios el código y en otras está bien visto, he clasificado las dos soluciones (una con while y una con do-while) que he diseñado, la primera de ellas con comentarios demostrando mi entendimiento sobre el código y la otra sin los comentarios, ya que soy muy flexible y siempre me adapto a lo que la empresa necesita, no a mis caprichos personales.
 - **menú de 4 opciones con bucles while:** Siendo esta la solución más "larga" porque tiene comentarios explicativos, con un total de **155 líneas**.

```
<?php
// Creación de las variables utilizadas en el ejercicio al
principio del script.
$opcionMenu;

while ($opcionMenu != 4) {
    // Función para mostrar el menú de opciones del programa.
    MostrarMenu();
    // Solicitud de una opción del menú con "readline" y "prompt".
    $opcionMenu = readline("Elige una opción: ");

    switch ($opcionMenu) {
        case 1:
            CalcularAreaRectangulo();
            break;
        case 2:
            ConvertirTemperaturaCF();
            break;
        case 3:
            DiferenciaTiempoFechas();
            break;
        case 4:
            echo "¡Gracias por usar este programa! Hasta luego.";
            break;
        default:
            echo "¡Has elegido una opción no valida! Intentalo de
nuevo.";
            break;
    };
};
```

```
/**
 * Función auxiliar que valida si un valor es un número.
 * @return int
 */
function ValidarNumero($mensaje)
{
    $valor = null;

    while (!is_numeric($valor)) {
        $valor = readline($mensaje);

        if (!is_numeric($valor)) {
            echo "¡Error! El valor debe ser un número, intentalo de
nuevo. \n";
        }
    }

    return (int)$valor;
}

/**
 * Función auxiliar que valida que un valor sea un número positivo
mayor a cero.
 * @return int
 */
function ValidarNumeroPositivo($mensaje)
{
    $valor = null;

    while (!is_numeric($valor) || $valor <= 0) {
        $valor = readline($mensaje);

        if (!is_numeric($valor) || $valor <= 0) {
            echo "¡Error! El valor debe ser un número positivo,
intentalo de nuevo. \n";
        }
    }

    return (int) $valor;
}

/**
 * Función encargada de calcular el área de un rectángulo (en
centímetros cuadrados) usando la base y altura
 * (en centímetros) del mismo.
 * @return void
 */
function CalcularAreaRectangulo()
{
    // Solicitud, intercepción y validación de datos con función
auxiliar.
    $base = ValidarNumeroPositivo("¿Cual es la base (en
centímetros) del rectángulo?: ");
    $altura = ValidarNumeroPositivo("¿Cual es la altura (en
centímetros) del rectángulo?: ");
}
```

```
// Logica de la función.
$area = $base * $altura;

// Imprimir resultado del calculo.
echo "El área de un rectángulo cuya base es de {$base}
centímetros y cuya altura es de {$altura} centímetros es {$area}
centímetros cuadrados. \n";
}

/**
 * Función que convierte una "X" temperatura dada en centigrados a
fahrenheit.
 * @return void
 */
function ConvertirTemperaturaCF()
{
    // Solicitud e intercepción de datos.
    $centigrados = ValidarNumero("¿Cuantos grados centigrados
deseas convertir a fahrenheit?: ");

    // Logica de la función.
    $fahrenheit = (($centigrados * 9) / 5) + 32;

    // Imprimir resultado del calculo.
    echo "{$centigrados} grados centigrados son {$fahrenheit} grados
fahrenheit. \n";
}

/**
 * Función encargada de recibir, validar y mostrar la diferencia en
días entre dos fechas.
 * @return void
 */
function DiferenciaTiempoFechas() {
    // Solicitud e intercepción de datos.
    $fechaA = ValidarFecha("Introduce la primera fecha
(DD-MM-YYYY): ");
    $fechaB = ValidarFecha("Introduce la segunda fecha
(DD-MM-YYYY): ");

    // Logica delegada a función auxiliar reutilizable.
    $diferencia = $fechaA->diff($fechaB);

    // Imprimir resultado del calculo.
    echo "La diferencia entre las fechas es de {$diferencia->days}
días.\n";
}

/**
 * Función encargada de validar que una fecha cualquiera ha sido
escrita correctamente, donde
 * correcto se entienden ambos, el formato internacional ISO-8601
(YYYY-MM-DD) y el formato
 * tradicional español (DD-MM-YYYY).
 * @param string $fecha
 * @return DateTime
 */
```

```
function ValidarFecha($mensaje) {
    do {
        // Solicitud e intercepción de datos.
        $fechaCruda = readline($mensaje);

        // Intento de convertir el String en una instancia de
        DateTime usando formato "d-m-Y".
        $fechaFinal = DateTime::createFromFormat("d-m-Y",
        $fechaCruda);

        // Si la conversión falla devuelve "false", ejecuta el "if"
        y el bucle continua.
        if (!$fechaFinal) {
            echo "¡Error! La fecha debe tener formato DD-MM-YYYY.
            \n";
        }

    } while (!$fechaFinal);

    return $fechaFinal;
}

/**
 * Función que muestra con "echo" las opciones del menú del script.
 * @return void
 */
function MostrarMenu()
{
    echo "
    ¡Elige una de las siguientes opciones! \n
    1.) Calcular área de un rectángulo \n
    2.) Convertir temperatura de centígrados a fahrenheit. \n
    3.) Calcular la distancia (en tiempo) entre dos fechas dadas.
    \n
    4.) Salir del programa. \n \n";
}
```

- o **menú de 4 opciones con bucles do-while:** Es exactamente el mismo programa con el mismo funcionamiento pero implementando bucles **do-while** en lugar de **while**, además, al no tener comentarios el tamaño se reduce a **108 líneas**:

```
<?php
$opcionMenu;

do {
    MostrarMenu();
    $opcionMenu = readline("Elige una opción: ");

    switch ($opcionMenu) {
        case 1:
            CalcularAreaRectangulo();
            break;
        case 2:
            ConvertirTemperaturaCF();
            break;
        case 3:
            DiferenciaTiempoFechas();
            break;
    }
}
```

```
        case 4:
            echo "¡Gracias por usar este programa! Hasta luego.";
            break;
        default:
            echo "¡Has elegido una opción no valida! Intentalo de nuevo.";
            break;
    };
} while ($opcionMenu != 4);

function ValidarNumero($mensaje)
{
    $valor = null;

    do {
        $valor = readline($mensaje);

        if (!is_numeric($valor)) {
            echo "¡Error! El valor debe ser un número, intentalo de nuevo. \n";
        }
    } while (!is_numeric($valor));

    return (int)$valor;
}

function ValidarNumeroPositivo($mensaje)
{
    $valor = null;

    do {
        $valor = readline($mensaje);

        if (!is_numeric($valor) || $valor <= 0) {
            echo "¡Error! El valor debe ser un número positivo, intentalo de nuevo. \n";
        }
    } while (!is_numeric($valor) || $valor <= 0);

    return (int) $valor;
}

function CalcularAreaRectangulo()
{
    $base = ValidarNumeroPositivo("¿Cual es la base (en centímetros) del rectangulo?: ");
    $altura = ValidarNumeroPositivo("¿Cual es la altura (en centímetros) del rectangulo?: ");

    $area = $base * $altura;

    echo "El área de un rectángulo cuya base es de {$base} centímetros y cuya altura es de {$altura} centímetros es {$area} centímetros cuadrados. \n";
}

function ConvertirTemperaturaCF()
{
    $centigrados = ValidarNumero("¿Cuantos grados centigrados deseas convertir a fahrenheit?: ");
```

```
$fahrenheit = (($centigrados * 9) / 5) + 32;

    echo "{$centigrados} grados centigrados son {$fahrenheit} grados
fahrenheit. \n";
}

function DiferenciaTiempoFechas() {
    $fechaA = ValidarFecha("Introduce la primera fecha (DD-MM-YYYY) :
");
    $fechaB = ValidarFecha("Introduce la segunda fecha (DD-MM-YYYY) :
");

    $diferencia = $fechaA->diff($fechaB);

    echo "La diferencia entre las fechas es de {$diferencia->days}
días.\n";
}

function ValidarFecha($mensaje) {
    do {
        $fechaCruda = readline($mensaje);

        $fechaFinal = DateTime::createFromFormat("d-m-Y",
$fechaCruda);

        if (!$fechaFinal) {
            echo "¡Error! La fecha debe tener formato DD-MM-YYYY. \n";
        }

    } while (!$fechaFinal);

    return $fechaFinal;
}

function MostrarMenu()
{
    echo "
    ¡Elige una de las siguientes opciones! \n
    1.) Calcular área de un rectángulo \n
    2.) Convertir temperatura de centigrados a fahrenheit. \n
    3.) Calcular la distancia (en tiempo) entre dos fechas dadas. \n
    4.) Salir del programa. \n \n";
}
```

Aunque pueda parecerlo **no he utilizado agentes de IA para realizar estos ejercicios**, tampoco considero que sea algo malo utilizarlos, todo lo contrario, ayudan a agilizar cosas, lo cierto es que en mis prácticas usaba **Claude** pero jamás pasándole código real de la empresa, lo que hacía era preguntarle “Si tuvieras que hacer esto, ¿Cómo lo harías?”, no le daba el código de la empresa como contexto, veía su respuesta y usándola como inspiración (solo si realmente entendía lo que estaba viendo) hacía mi propia versión.

De esa forma jamás expongo ni alimento agentes de **IA** con código real, mucho menos el de mis empleadores, pero puedo dejar que me de ideas, lo mismo que hago cuando por

ejemplo navegar en internet y visito foros como **Reddit** o **StackOverFlow**, jamás copiar y pegar, siempre entender y rediseñar.

El **PHPDoc** me gusta usarlo en PHP nativo porque a través del tipo de argumentos y returns de funciones/métodos tengo acceso al autocompletado de métodos de los tipos específicos que uso así como mejores advertencias y señalamiento de errores del **IDE** cuando, por ejemplo, detecta que en teoría quiero que mi función devuelva un **String** y lo que devuelve realmente es, por ejemplo, un **int**.

4. Programación orientada a objetos

- **Pregunta:** Crea una jerarquía de clases para representar diferentes tipos de vehículos (coches, motos, camiones). Cada vehículo debe tener propiedades como marca, modelo y año, y métodos como arrancar() y acelerar(). Además, los camiones deben tener una propiedad adicional para indicar su carga máxima.

Implementa un método estático en la clase base "Vehiculo" llamado "crearVehiculo()" que tome como parámetro un array asociativo con los datos del vehículo y devuelva un objeto de la clase correspondiente (coche, moto o camión).

- **Respuesta:**
 - **Clase Vehículo:**

```
<?php
/**
 * Objetivo: Crea una jerarquía de clases para representar diferentes
 tipos de vehículos (coches, motos, camiones).
 * Cada vehículo debe tener propiedades como marca, modelo y año, y
 métodos como arrancar() y acelerar(). Además,
 * los camiones deben tener una propiedad adicional para indicar su
 carga máxima.
 *
 * Implementa un método estático en la clase base "Vehiculo" llamado
 "crearVehiculo()" que tome como parámetro un
 * array asociativo con los datos del vehículo y devuelva un objeto de
 la clase correspondiente
 * (coche, moto o camión).
 *
 * La clase es abstracta porque no debería crear instancias por si
 sola, si no, servir como base para la creación
 * de instancias de sus clases hijas.
 */
abstract class Vehiculo
{
    // Campos necesarios para la creación de un vehiculo según el
 enunciado.
    private $marca;
    private $modelo;
    private $año;
```

```
// Mi valor añadido es un contador de instancias para simular los
ID's unicos de cada instancia.
private $id;
private static $contador = 1;

/**
 * Constructor de la clase "Vehiculo" que permite instanciar
vehiculos.
 * @param string $marca
 * @param string $modelo
 * @param string $anyo
 */
public function __construct($marca, $modelo, $anyo)
{
    // Primero asigna el valor, luego incrementa.
    $this->id = self::$contador++;
    $this->marca = $marca;
    $this->modelo = $modelo;
    $this->anyo = $anyo;
}

/**
 * Función "arrancar" que devuelve un String con un mensaje
especifico de la instancia
 * que lo invoca.
 * @return string
 */
public function arrancar()
{
    return "El vehículo {$this->marca} {$this->modelo} está
arrancando...";
}

/**
 * Función "acelerar" que devuelve un String con un mensaje
especifico de la instancia
 * que lo invoca.
 * @return string
 */
public function acelerar()
{
    return "El vehículo {$this->marca} {$this->modelo} está
acelerando...";
}

/**
 * Función estatica de de "Vehiculo", recibe en forma de Array los
datos del
 * vehiculo y retorna una instancia de la clase que lo
invoca/utiliza.
 * @param array $datos
 * @return Camion|Coche|Moto
 */
public static function crearVehiculo($datos)
{
    // Valor añadido, ¿No existe el campo "datos" en el Array?
Lanzo una excepción.
    if (!isset($datos["tipo"])) {
        throw new \Exception("¡Excepcion! No existe el campo
\"tipo\" en el Array de datos.");
    }
}
```

```

    }

    // A través de "tipo" se a que clase pertenece la instancia.
    $tipo = $datos["tipo"];
    // Campos comunes de todos los vehiculos heredados de la clase
    Vehiculo.
    $marca = $datos["marca"] ?? "";
    $modelo = $datos["modelo"] ?? "";
    $anyo = $datos["anyo"] ?? date("Y");

    // Dependiendo de la clase que invoca se ejecuta una función u
    otra para instanciar.
    switch ($tipo) {
        case "coche":
            return new Coche($marca, $modelo, $anyo);
        case "moto":
            return new Moto($marca, $modelo, $anyo);
        case "camion":
            $cargaMaxima = $datos["cargaMaxima"] ?? 0;
            return new Camion($marca, $modelo, $anyo,
            $cargaMaxima);
        // Valor añadido hipotetico si se intenta instanciar desde
        un tipo no valido.
        default:
            throw new \Exception("¡Excepción! Tipo de vehículo no
            válido, se ha intentando instanciar usando el siguiente tipo:
            {$tipo}");
    }
}
}

```

- Clase Coche:

```

<?php
/**
 * Clase "Coche", hereda todos los métodos/funciones de la clase
 Vehiculo.
 */
class Coche extends Vehiculo {}

```

- Clase Moto:

```

<?php
/**
 * Clase "Moto", hereda todos los métodos/funciones de la clase
 Vehiculo.
 */
class Moto extends Vehiculo {}

```

- Clase Camión:

```

<?php
/**
 * Clase "Camion", hereda todos los métodos/funciones de la clase
 Vehiculo con un tipo de campo/propiedad propio.
 */
class Camion extends Vehiculo
{

```

```
// Variable propia de la clase que define la carga maxima de un
camión cualquiera.
private $cargaMaxima;
/**
 * Constructor personalizado de camión que hereda el constructor
padre de la
 * clase vehiculo y añade la propiedad propia de la clase al
mismo.
 * @param int $cargaMaxima
 */
public function __construct($marca, $modelo, $anyo, $cargaMaxima)
{
    parent::__construct($marca, $modelo, $anyo);
    $this->cargaMaxima = $cargaMaxima;
}
}
```

- o Clase Main (pruebas):

```
<?php
// Solo necesitamos cargar una vez las clases para usarlas.
require_once "Vehiculo.php";
require_once "Coche.php";
require_once "Moto.php";
require_once "Camion.php";

// Crear un coche usando el método estático crearVehiculo().
$coche = Vehiculo::crearVehiculo([
    "tipo" => "coche",
    "marca" => "Toyota",
    "modelo" => "Corolla",
    "anyo" => 2023
]);
echo "Coche creado: {$coche->arrancar()} \n";
echo "Coche: {$coche->acelerar()} \n";

// Crear una moto usando el método estático crearVehiculo().
$moto = Vehiculo::crearVehiculo([
    "tipo" => "moto",
    "marca" => "Honda",
    "modelo" => "CBR 600",
    "anyo" => 2022
]);
echo "Moto creada: {$moto->arrancar()} \n";
echo "Moto: {$moto->acelerar()} \n";

// Crear un camión usando el método estático crearVehiculo().
$camion = Vehiculo::crearVehiculo([
    "tipo" => "camion",
    "marca" => "Volvo",
    "modelo" => "FH16",
    "anyo" => 2021,
    "cargaMaxima" => 25000
]);
echo "Camión creado: {$camion->arrancar()} \n";
echo "Camión: {$camion->acelerar()} \n";
```

Sección: Bash

5. Conexión a servidores remotos y edición de archivos

- **Pregunta:** ¿Cómo te conectarías a un servidor remoto vía SSH y cómo modificarías un archivo de configuración en ese servidor?
- **Respuesta:** Durante mis prácticas realice conexiones utilizando **FileZilla**, pero en **sistemas informáticos** (donde tuve uno de los mejores profesores que jamás he tenido) y en **despliegue** (otra asignatura con un excelente profesor) realizamos pruebas locales, probablemente la secuencia sería la siguiente:
 - **Conexión el comando “ssh -p 1406 [usuario@servidor.com](#)”:** Normalmente en las empresas los puertos no se dejan por defecto, así que en el comando se utiliza el atributo “-p” para especificar el puerto.
 - **Autenticación de identidad:** En las pruebas locales en mi formación utilizamos una contraseña sin cifrado y en una práctica utilizamos cifrado con “ssh-keygen”.
 - **Conectado (hipotéticamente):** Normalmente realizamos copias de todos los documentos previamente a modificarlos, trabaje con archivos con contenido en forma de texto sobre todo utilizando **nano** para las ediciones por ejemplo.

Estoy es muy susceptible al contexto, ¿puedo instalar **ssh2** como extensión de PHP para crear funciones y modificar/acceder a ficheros desde un entorno **PHP**? ¿La empresa utiliza **FileZilla** o parecidos? Pero soy una persona muy autodidacta y los conceptos de conexión/desconexión y creación/lectura/actualización/eliminación (**CRUD**) son esencialmente los mismos, así que me atrevo a decir que sería capaz de adaptarme a cualquier escenario.

6. Scripts básicos

- **Pregunta:** Escribe un script en Bash que recorra todos los archivos de un directorio y los copie a otro directorio, omitiendo los directorios.
- **Respuesta:** Probablement sería algo así (no soy experto en bash pero ha sido ponerme hoy a hacer un cursito de bash para reforzar):

```
origen="$1"
destino="$2"

for archivo in "$origen"/*; do
    if [ -f "$archivo" ]; then
        cp "$archivo" "$destino"
    fi
done
```

Explicación: En bash existen las “variables especiales”, cuando queremos pasarle “X” argumentos a un script se utilizan, “\$1” es el primer argumento, “\$2” el segundo argumento, es el equivalente en **PHP** a un **function script (\$origen, \$destino) {}**, cuando ejecutemos este **script** (imaginemos que se llama “[nextlane.sh](#)” tendríamos que ejecutarlo de la siguiente

forma pasando los argumentos: **"script.sh /home/agu1406/documentos /home/usuario/documentoscopia"**. (dos rutas como argumento).

Ahora, en la variable **"archivo"** se guardan ambos, directorios y archivos, el filtro **"-f"** del if es el encargado de **"Sí es un archivo (file) entonces copialo"**, omitiendo así los directorios.

Sección: MySQL

7. Consulta básica con filtro y ordenamiento

- **Pregunta:** Escribe una consulta SQL para seleccionar todos los usuarios de una tabla "usuarios" cuyo nombre empiece por "A" y ordenarlos por apellido.

Tabla "usuarios":

Id_contacto	nombre	apellido	email
1	Ana	García	ana_garcia@correo.es
2	Beatriz	Pérez	Beatriz.perez@correo.es
3	Carlos	López	carlosL@correo.es
4	Álvaro	Martínez	A.Martinez@correo.es
5	Andrea	Sánchez	Andrea-sanchez@correo.es

- **Respuesta:**

```
SELECT *  
FROM usuarios  
WHERE nombre LIKE 'A%'  
ORDER BY apellido;
```

8. Unión de tablas

- **Pregunta:** Tienes dos tablas: "pedidos" y "productos". Realiza una consulta que muestre el nombre del producto y el nombre del cliente para cada pedido.

Tabla "pedidos":

id_pedido	id_producto	id_cliente	fecha
1	10	1	2023-11-20
2	12	2	2023-11-21
3	10	3	2023-11-22

Tabla "productos"

id_producto	nombre_producto	precio
10	Teléfono móvil	300
11	Tablet	450
12	Auriculares	50

Tabla "Clientes"

id_cliente	nombre_cliente
1	Juan Pérez
2	María López
3	Pedro García

- Respuesta:

```
SELECT
    productos.nombre_producto,
    clientes.nombre_cliente
FROM pedidos
INNER JOIN productos ON pedidos.id_producto = productos.id_producto
INNER JOIN clientes ON pedidos.id_cliente = clientes.id_cliente;
```

9. Agrupación y funciones de agregación

Pregunta: Calcula el número total de pedidos por cliente y muestra el cliente con más pedidos.

Tabla "pedidos": (misma que en el ejemplo anterior)

- Respuesta:

```
SELECT
    id_cliente,
    COUNT(*) AS total_pedidos
FROM pedidos
GROUP BY id_cliente
ORDER BY total_pedidos DESC
LIMIT 1;
```

Sección Git

10. Fundamentos

- Pregunta: ¿Cuál es la diferencia entre git fetch y git pull? ¿Cuándo usarías uno u otro?
- Respuesta:
 - **git pull**: Esto es peligroso si no se usa con cuidado, mira la rama remota, descarga todos los cambios y hace **merge** de los mismos con tu repositorio local, siempre se deben tener los últimos cambios en las ramas al trabajar en ellas para evitar conflictos, pero aprendí (por las malas, pero así es como mejor se aprende) que lo mejor es **primero mirar los cambios** antes de hacer **merge** de los mismos en la rama donde se esté trabajando.
 - **git fetch**: Por otra parte, esto es genial, baja los cambios en local pero no hacer **merge** de los mismos si no que es posible verlos antes de integrarlos a nuestro trabajo local.

"El código que escribes hoy es el código que tendrás que mantener mañana" - Anónimo.